



## Carátula para entrega de prácticas

Facultad de Ingeniería

Laboratorios de docencia

# Laboratorio de Computación Salas A y B

*Profesor(a):* Karina García Morales

*Asignatura:* Fundamentos de Programación

*Grupo:* 22

*No de Práctica(s):* Practica 9

*Integrante(s):* Roja Martínez Francisco Alexander

*No. de lista o  
brigada:* 40

*Semestre:* 2026-1

*Fecha de entrega:*

*Observaciones:*

## CALIFICACIÓN:

## *Arreglos unidimensionales*

### ***Objetivo:***

El alumno utilizará arreglos de una dimensión en la elaboración de programas que resuelvan problemas que requieran agrupar datos del mismo tipo, alineados en un vector o lista.

### ***Introducción:***

Un arreglo es, fundamentalmente, una colección de datos que comparten el mismo tipo (por ejemplo, solo números enteros o solo caracteres) y que están almacenados de forma contigua en la memoria. Es crucial definir su tamaño cuando se crea, ya que este no puede cambiar posteriormente.

Para acceder a un elemento específico dentro del arreglo, necesitamos su índice, que es la posición asociada a ese dato particular. La dimensión de un arreglo se define por cuántos índices necesitas para localizar un elemento. Usamos arreglos para simplificar y hacer más eficiente el código al trabajar con muchos datos que tienen una relación o un propósito común.

### ***Arreglos Unidimensionales***

Un arreglo unidimensional es como una lista lineal de datos que se guardan uno junto al otro en la memoria de la computadora.

Almacenamiento y Posiciones: Si el arreglo tiene un tamaño de 'n' elementos, la primera posición se llama índice 0, y la última posición válida es el índice n-1.

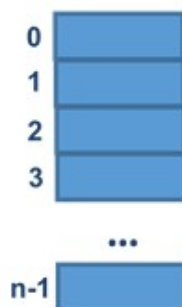


Figura 1. Almacenamiento en memoria de un arreglo unidimensional

Declaración en C: Para crear un arreglo en el lenguaje C, se usa una sintaxis simple:

tipoDeDato nombre[tamaño]

tipoDeDato: Define si los elementos serán enteros (int), reales (float), caracteres (char), o cualquier otro tipo, incluso estructuras.

nombre: Es el identificador que usas para referirte al arreglo.

tamaño: Es un número entero fijo que indica la cantidad máxima de elementos que puede contener esa lista.

### ***Código (arreglo unidimensional empleando la estructura while)***

A continuación, se muestra el código de un programa que genera un arreglo unidimensional de 5 elementos y que para poder acceder, recorrer y mostrar cada elemento del arreglo se usa la variable índice desde 0 hasta 4 haciendo uso de un ciclo while.

Programala.c

```
#include <stdio.h>
int main ()
{
    int lista[5] = {10, 8, 5, 8, 7}; // Se declara e inicializa el arreglo unidimensional
    int indice = 0;
    printf("\tLista\n");
    while (indice < 5 ) // Acceso a cada elemento del arreglo unidimensional usando while
    {
        printf("\nCalificación del alumno %d es %d", indice+1, lista[indice]);
        indice += 1;      // Sentencia análoga a indice = indice + 1;
    }

    printf("\n");
    return 0;
}
```

### ***Código (arreglo unidimensional empleando la estructura do-while)***

En el siguiente código se genera un arreglo unidimensional de 5 elementos y que para poder acceder, recorrer y mostrar cada elemento del arreglo se usa la variable índice desde 0 hasta 4 haciendo uso de un ciclo do-while.

#### Programa1b.c

```
#include <stdio.h>
int main ()
{
    int lista[5] = {10, 8, 5, 8, 7}; // Se declara e inicializa el arreglo unidimensional
    int indice = 0;
    printf("\tLista\n");
    do // Acceso a cada elemento del arreglo unidimensional usando do-while
    {
        printf("\nCalificación del alumno %d es %d", indice+1, lista[indice]);
        indice += 1; // Sentencia análoga a indice = indice + 1;
    }
    while (indice < 5 );
    printf("\n");
    return 0;
}
```

Código (arreglo unidimensional empleando la estructura for) A continuación, se muestra el código que genera un arreglo unidimensional de 5 elementos y que para poder acceder, recorrer y mostrar cada elemento del arreglo se usa la variable indice desde 0 hasta 4 haciendo uso de un ciclo for.

#### Programa1c.c

```
#include <stdio.h>
int main ()
{
    int lista[5] = {10, 8, 5, 8, 7}; // Se declara e inicializa el arreglo unidimensional
    int indice=0;
    printf("\tLista\n");
    // Acceso a cada elemento del arreglo unidimensional usando for
    for (indice = 0 ; indice < 5 ; indice++)
    {
        printf("\nCalificación del alumno %d es %d", indice+1, lista[indice]);
    }

    printf("\n");
    return 0;
}
```

### *Apuntadores*

Un apuntador es una variable especial que no almacena un valor directo, sino la dirección de memoria de otra variable. Piensa en él como una flecha que "apunta" a dónde se encuentra un dato.

Trabajar con apuntadores nos permite acceder y manipular datos en la memoria de forma muy rápida y eficiente.

### ***Declaración y Asignación***

La sintaxis para declarar un apuntador y asignarle una dirección es la siguiente:

1. Declaración: Se usa el asterisco (\*) antes del nombre para indicar que es un apuntador.

```
textTipoDeDato *apuntador, variable;
```

2. Asignación de Dirección: Se usa el ampersand (&) para obtener la dirección de memoria de la variable (esto es lo mismo que se hace con scanf).

```
apuntador = &variable;
```

Nota Importante: Un apuntador solo puede almacenar la dirección de una variable que sea del mismo textTipoDeDato con el que fue declarado.

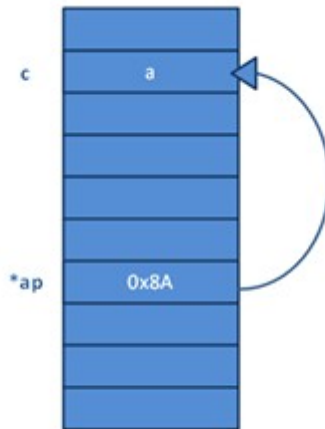


Figura 2. Un apuntador almacena la dirección de memoria de la variable a la que apunta

### ***Apuntadores y su relación con los arreglos.***

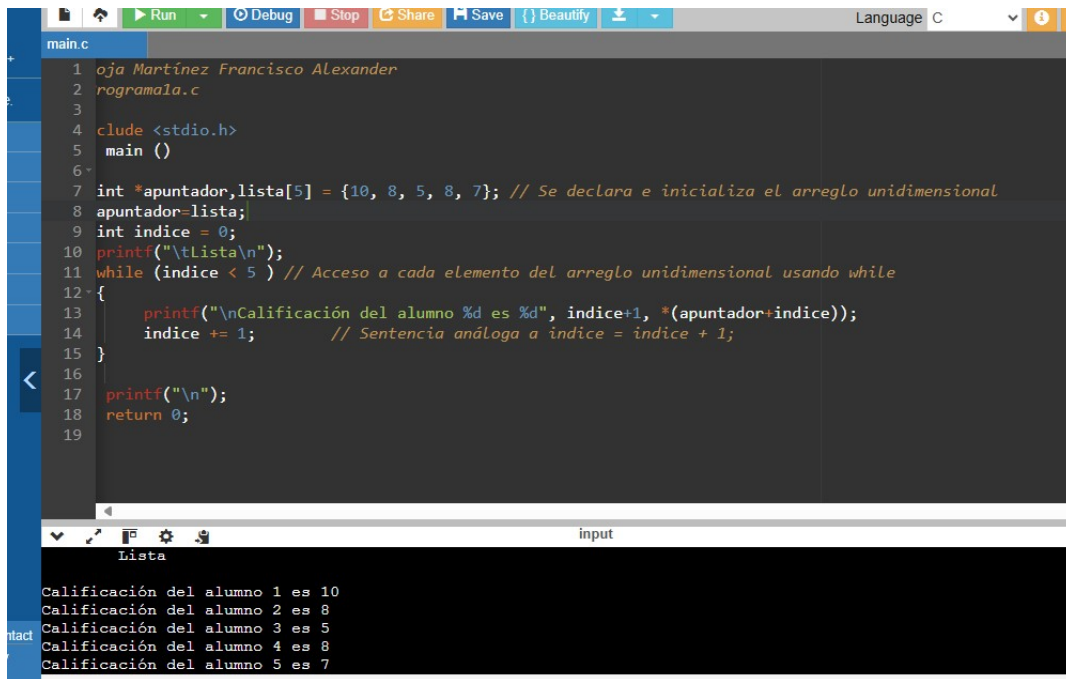
Cabe mencionar que el nombre de un arreglo es un apuntador fijo al primero de sus elementos; por lo que las siguientes instrucciones, para el código de arriba, son equivalentes:

apEnt = &c[0];

apEnt = c;

### *Trabajo en clase*

### *3. Ejecutamos programa 1a.c y lo modificamos:*



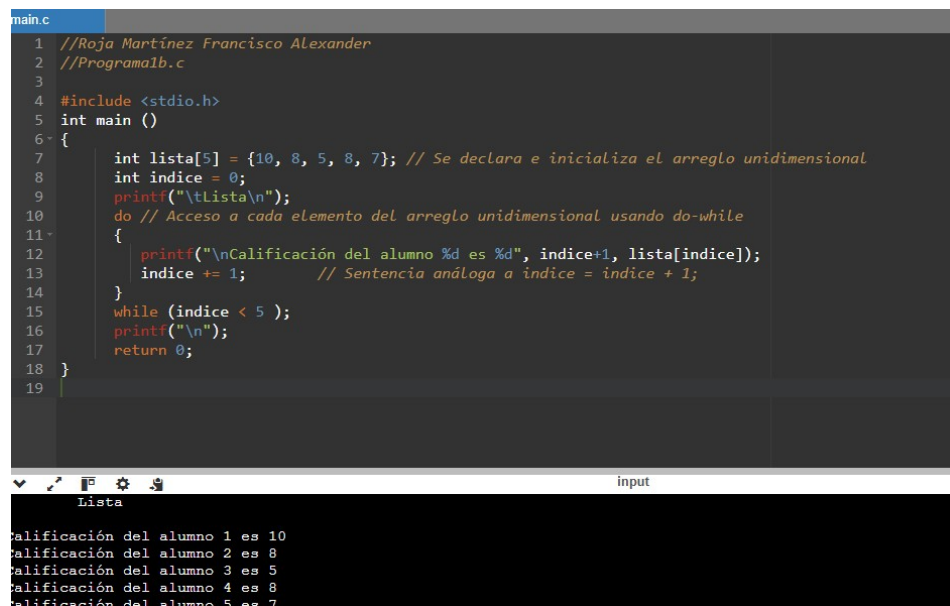
```
1 oja Martínez Francisco Alexander
2 rogramala.c
3
4 clude <stdio.h>
5 main ()
6
7 int *apuntador, lista[5] = {10, 8, 5, 8, 7}; // Se declara e inicializa el arreglo unidimensional
8 apuntador=lista;
9 int indice = 0;
10 printf("\tLista\n");
11 while (indice < 5 ) // Acceso a cada elemento del arreglo unidimensional usando while
12 {
13     printf("\nCalificación del alumno %d es %d", indice+1, *(apuntador+indice));
14     indice += 1; // Sentencia análoga a indice = indice + 1;
15 }
16
17 printf("\n");
18 return 0;
19
```

input

Lista

Calificación del alumno 1 es 10  
Calificación del alumno 2 es 8  
Calificación del alumno 3 es 5  
Calificación del alumno 4 es 8  
Calificación del alumno 5 es 7

### *Se revisa el cambio del primer programa con do-while y for*

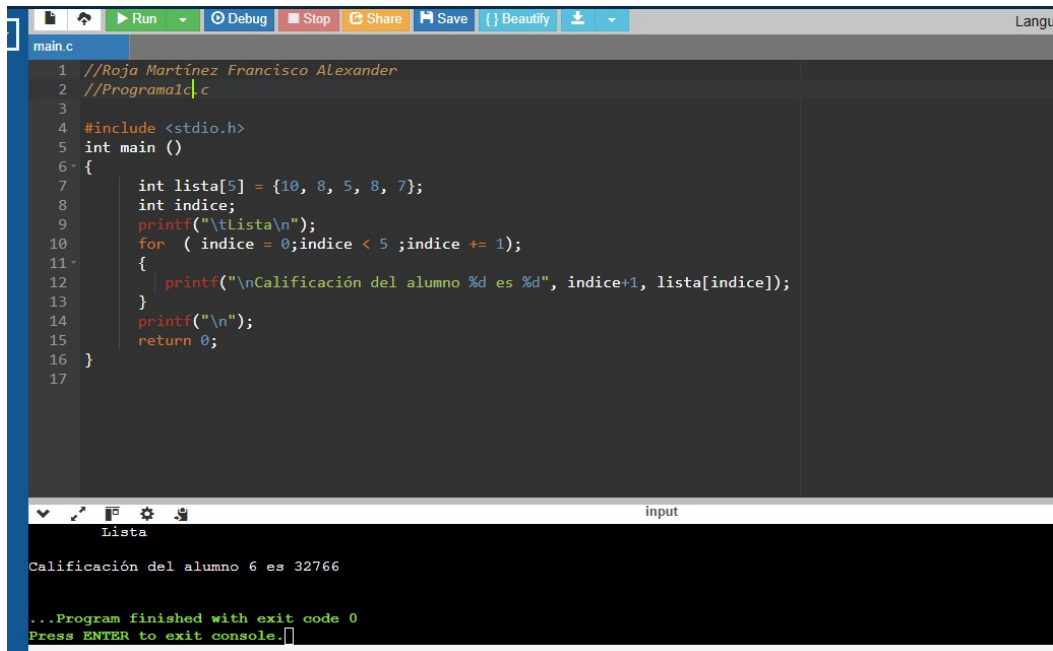


```
1 //Rojá Martínez Francisco Alexander
2 //Programal1b.c
3
4 #include <stdio.h>
5 int main ()
6 {
7     int lista[5] = {10, 8, 5, 8, 7}; // Se declara e inicializa el arreglo unidimensional
8     int indice = 0;
9     printf("\tLista\n");
10    do // Acceso a cada elemento del arreglo unidimensional usando do-while
11    {
12        printf("\nCalificación del alumno %d es %d", indice+1, lista[indice]);
13        indice += 1; // Sentencia análoga a indice = indice + 1;
14    }
15    while (indice < 5 );
16    printf("\n");
17    return 0;
18 }
19
```

input

Lista

Calificación del alumno 1 es 10  
Calificación del alumno 2 es 8  
Calificación del alumno 3 es 5  
Calificación del alumno 4 es 8  
Calificación del alumno 5 es 7



```
1 //Rojá Martínez Francisco Alexander
2 //Programa1.c
3
4 #include <stdio.h>
5 int main ()
6 {
7     int lista[5] = {10, 8, 5, 8, 7};
8     int indice;
9     printf("\tLista\n");
10    for ( indice = 0; indice < 5 ; indice += 1);
11    {
12        printf("\nCalificación del alumno %d es %d", indice+1, lista[indice]);
13    }
14    printf("\n");
15    return 0;
16 }
17
```

input

Lista

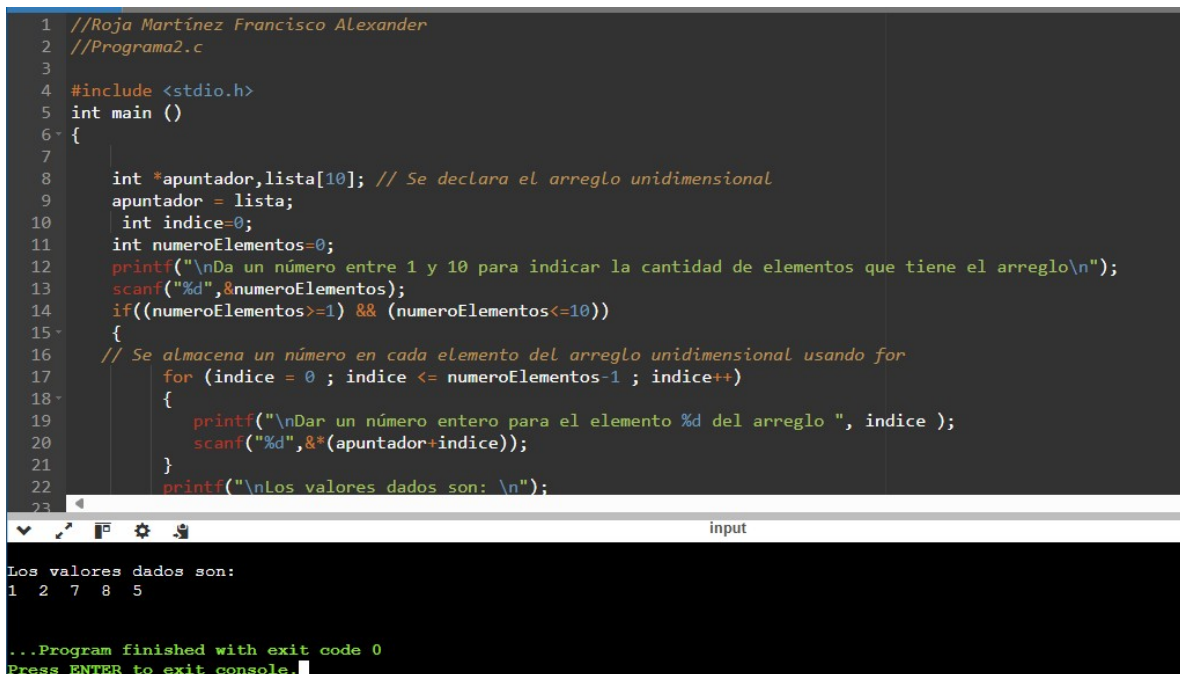
Calificación del alumno 6 es 32766

...Program finished with exit code 0  
Press ENTER to exit console.

#### 4. ¿Que hace el programa 2.c?

Este programa tiene como objetivo principal declarar, llenar y luego mostrar los elementos de un arreglo unidimensional llamado lista, con una característica particular: el usuario decide cuántos elementos (entre 1 y 10) desea ingresar.

#### 5. Al programa2.c le vamos a aplicar apunadores



```
1 //Rojá Martínez Francisco Alexander
2 //Programa2.c
3
4 #include <stdio.h>
5 int main ()
6 {
7
8     int *apuntador, lista[10]; // Se declara el arreglo unidimensional
9     apuntador = lista;
10    int indice=0;
11    int numeroElementos=0;
12    printf("\nDa un número entre 1 y 10 para indicar la cantidad de elementos que tiene el arreglo\n");
13    scanf("%d",&numeroElementos);
14    if((numeroElementos>=1) && (numeroElementos<=10))
15    {
16        // Se almacena un número en cada elemento del arreglo unidimensional usando for
17        for (indice = 0 ; indice <= numeroElementos-1 ; indice++)
18        {
19            printf("\nDar un número entero para el elemento %d del arreglo ", indice );
20            scanf("%d",&(apuntador+indice));
21        }
22        printf("\nLos valores dados son: \n");
23    }

```

input

Los valores dados son:

1 2 7 8 5

...Program finished with exit code 0  
Press ENTER to exit console.

## **6. Describe la sintaxis del arreglo**

La sintaxis utilizada en el programa para declarar el arreglo unidimensional es:

```
tipoDeDato nombre [tamaño];
```

En este programa, la sintaxis específica en la línea 8 es:

```
int lista[10];
```

- int: Es el tipo de dato de los elementos del arreglo (enteros).
- lista: Es el nombre o identificador del arreglo.
- [10]: Es el tamaño fijo del arreglo, indicando que puede almacenar 10 elementos.

## **7. Dar un ejemplo de arreglo unidimensional(explicito o implícito)**

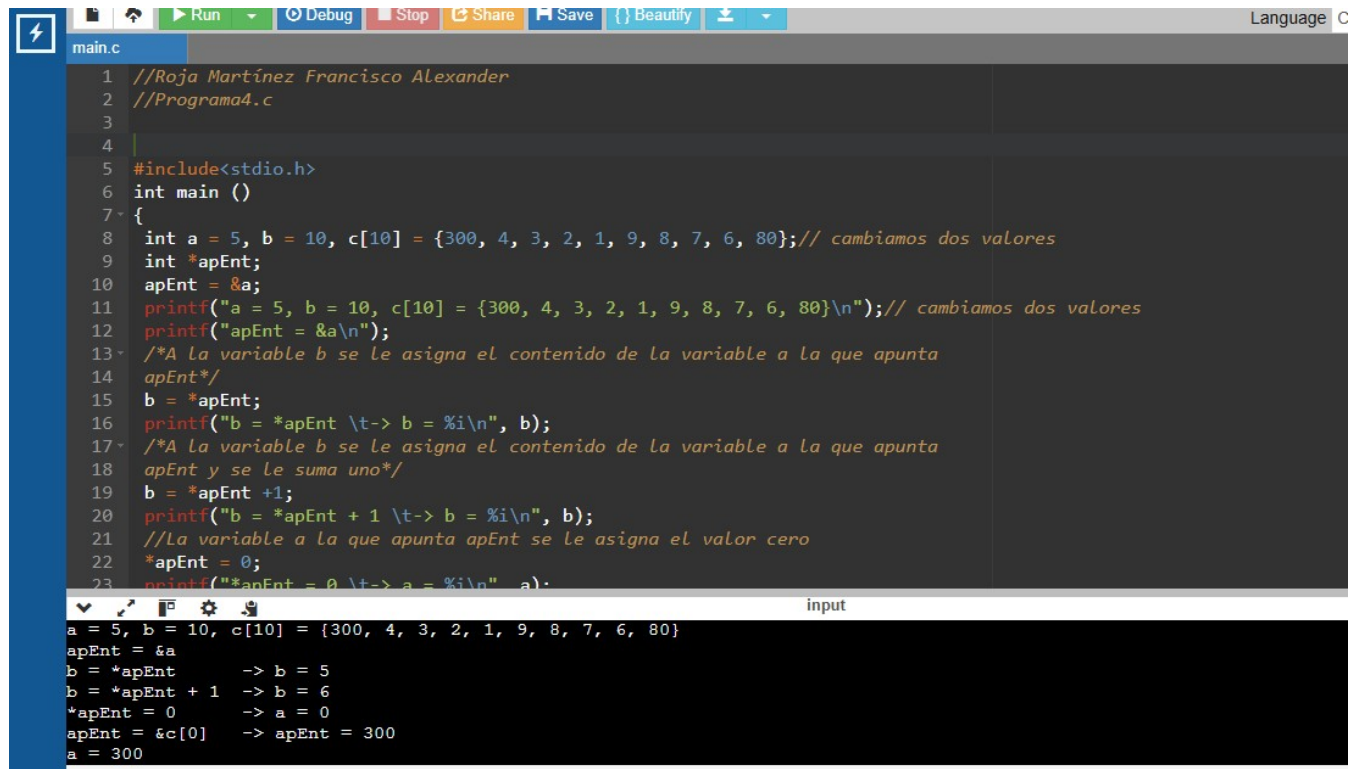
```
int edades[5]; // Declaración de un arreglo de 5 enteros.
```

## **8. ¿Qué estructura requieres para manipular un arreglo por medio de sus índices?**

Para manipular (recorrer, llenar o mostrar) todos los elementos de un arreglo de manera eficiente, accediendo a ellos uno por uno a través de sus índices, la estructura fundamental requerida es un Ciclo Repetitivo (o Bucle).



## 9. Ejecución de programa 4.c con modificaciones



```
1 //Rojá Martínez Francisco Alexander
2 //Programa4.c
3
4
5 #include<stdio.h>
6 int main ()
7 {
8     int a = 5, b = 10, c[10] = {300, 4, 3, 2, 1, 9, 8, 7, 6, 80}; // cambiamos dos valores
9     int *apEnt;
10    apEnt = &a;
11    printf("a = 5, b = 10, c[10] = {300, 4, 3, 2, 1, 9, 8, 7, 6, 80}\n"); // cambiamos dos valores
12    printf("apEnt = &a\n");
13    /*A la variable b se le asigna el contenido de la variable a la que apunta
14    apEnt*/
15    b = *apEnt;
16    printf("b = *apEnt \t-> b = %i\n", b);
17    /*A la variable b se le asigna el contenido de la variable a la que apunta
18    apEnt y se le suma uno*/
19    b = *apEnt + 1;
20    printf("b = *apEnt + 1 \t-> b = %i\n", b);
21    //La variable a la que apunta apEnt se le asigna el valor cero
22    *apEnt = 0;
23    printf("%apEnt = 0 \t-> a = %i\n", a);
```

input

```
a = 5, b = 10, c[10] = {300, 4, 3, 2, 1, 9, 8, 7, 6, 80}
apEnt = &a
b = *apEnt      -> b = 5
b = *apEnt + 1  -> b = 6
*apEnt = 0      -> a = 0
apEnt = &c[0]   -> apEnt = 300
a = 300
```

## 10. Programa5.c que indica como trabaja un apuntador recorriendo un arreglo

En el programa Programa5.c se ilustra cómo se utiliza un apuntador para recorrer y acceder a los elementos de un arreglo unidimensional mediante la técnica de aritmética de apuntadores.

El programa sigue estos pasos clave:

1. Declaración del Arreglo: Se declara e inicializa el arreglo `int arr[] = {75, 4, 3, 2, 1};`.
2. Declaración del Apuntador: Se declara `int *apArr;` como un apuntador a entero.
3. Asignación Base: La instrucción `apArr = arr;` asigna la dirección de memoria del primer elemento (`arr[0]`) al apuntador `apArr`.
4. Acceso Secuencial: El programa accede a cada elemento del arreglo usando la notación de apuntadores y la suma de enteros:
  - Primer elemento: `x = *apArr;` (Accede a `arr[0]`, que es 75).

- Sigüientes elementos:  $x = *(apArr + 1);$ ,  $x = *(apArr + 2);$ , etc. (Acceden a  $arr[1]$ ,  $arr[2]$ , etc.).

```

main.c
1 //Roja Martínez Francisco Alexander
2 //Programa5.c
3
4 #include <stdio.h>
5 int main ()
6 {
7     int arr[] = {75, 4, 3, 2, 1}; //tamaño 5 de forma implícita porque está inicializado
8     int *apArr; //Se declara el apuntador apArr
9     int x;
10    apArr = arr; //apArr = &arr[0];
11    printf("int arr[] = {75, 4, 3, 2, 1};\n");
12    printf("apArr = &arr[0]\n");
13    x = *apArr; /*A la variable x se le asigna el contenido del arreglo arr en su
14    elemento 0*/
15    printf("x = *apArr \t -> x = %d\n", x);
16    x = *(apArr+1); /*A la variable x se le asigna el contenido del arreglo arr
17    en su elemento 1*/
18    printf("x = *(apArr+1) \t -> x = %d\n", x);
19    x = *(apArr+2); /*A la variable x se le asigna el contenido del arreglo arr
20    en su elemento 2*/
21    printf("x = *(apArr+2) \t -> x = %d\n", x);
22    x = *(apArr+3); /*A la variable x se le asigna el contenido del arreglo arr
23    en su elemento 3*/

```

input

```

int arr[] = {75, 4, 3, 2, 1};
apArr = &arr[0]
x = *apArr      -> x = 75
x = *(apArr+1)  -> x = 4
x = *(apArr+2)  -> x = 3
x = *(apArr+3)  -> x = 2
x = *(apArr+4)  -> x = 1

```

## 11. Programa7.c (aplicamos el uso de Strlen)

```

main.c
1 //Roja Martínez Francisco Alexander
2 //Programa 7.c
3
4 #include <stdio.h>
5 #include <string.h>
6 int main()
7 {
8     char palabra[20]; //arreglo tamaño 20
9     int i=0;
10    printf("Ingrese una palabra: ");
11    scanf("%s", palabra); /* Se omite & porque el propio nombre del arreglo de
12    tipo cadena apunta, es decir, es equivalente a la dirección de comienzo del
13    propio arreglo*/
14    printf("La palabra ingresada es: %s\n", palabra); // si fuera carácter sería %c
15    // este for me sirve para imprimir la palabra que ingresó el usuario
16    for (i = 0 ; i<strlen(palabra) ; i++) // i < 20      i<strlen(palabra-1)  (n-1)
17    {
18        printf("%c", palabra[i]); // %c porque va a recorrer letra por letra
19    }
20    return 0;
21 }
22
23 // &lista[indice] -> &*(apuntador+indice) implementación de apuntadores

```

input

```

Ingrese una palabra: Alexander
La palabra ingresada es: Alexander
Alexander
...Program finished with exit code 0
Press ENTER to exit console.

```

## *Tarea*

**1.-Indica que realiza el siguiente programa:**

```
#include <stdio.h>

int arreglo[] = {16,32,42,67,25,20,73,28,17,45};

int i, j, n, aux;

int main() {
    n = 10;
    for(i=1; i< i++)
    {
        j = i;
        aux = arreglo[i];
        while(j>0 && aux<arreglo[j-1])
        {
            arreglo[j] = arreglo[j-1];
            j=j-1;
        }
        arreglo[j] = aux;
    }
    printf("\n\nLos elementos obtenidos del arreglo son: \n");
    for(i=0; i< i++)
    {
        printf("Elemento [%d]: %d\n", i, arreglo[i]);
    }
    return 0;
}
```

El propósito del código es ordenar los elementos del arreglo de forma ascendente.

El ciclo for externo recorre el arreglo a partir del segundo elemento (i=1).

Dentro del ciclo, aux guarda el elemento actual a insertar (arreglo[i]).

El ciclo while interno compara aux con los elementos ya ordenados a su izquierda (arreglo[j-1]).

Si el elemento a la izquierda es mayor que aux, lo desplaza una posición a la derecha (arreglo[j] = arreglo[j-1];).

Finalmente, cuando se encuentra la posición correcta, aux (el elemento original) se inserta en ese lugar (arreglo[j] = aux;).

### *Nota*

El código tiene errores de sintaxis en los ciclos for que impedirían su correcta ejecución:

Ciclo de Ordenamiento: for(i=1; i < i++) es incorrecto. Debería ser for(i=1; i < n; i++) o for(i=1; i < 10; i++). La condición i < i++ probablemente causará un bucle infinito o un comportamiento indefinido.

Ciclo de Impresión: for(i=0; i < i++) es incorrecto. Debería ser for(i=0; i < n; i++) o for(i=0; i < 10; i++).

Ya corregidos para ejecutarse correctamente (for(i=1; i < 10; i++) y for(i=0; i < 10; i++)), la salida del programa sería el arreglo ordenado de menor a mayor:

**{16, 17, 20, 25, 28, 32, 42, 45, 67, 73}**

```
1 //Rojá Martínez Francisco Alexander
2 //Tarea
3
4 #include <stdio.h>
5
6 // El arreglo y las variables se definen globalmente para simplicidad
7 int arreglo[] = {16, 32, 42, 67, 25, 20, 73, 28, 17, 45};
8 int i, j, n, aux;
9
10 int main() {
11     // n es el número total de elementos en el arreglo
12     n = 10;
13
14     // --- CICLO DE ORDENAMIENTO POR INSERCIÓN (CORREGIDO) ---
15     // i va desde el segundo elemento (índice 1) hasta el último (índice 9)
16     for(i = 1; i < n; i++)
17     {
18         j = i;
19         aux = arreglo[i]; // Guarda el valor a insertar
20
21         // Desplaza los elementos mayores a la derecha
22         while(j > 0 && aux < arreglo[j-1])
23         {
24             arreglo[j] = arreglo[j-1];
25             j--;
26         }
27         arreglo[j] = aux;
28     }
29
30     printf("Los elementos del arreglo ordenados son:\n");
31     for(i = 0; i < n; i++)
32     {
33         printf("Elemento [%d]: %d\n", i, arreglo[i]);
34     }
35 }
```

input

Los elementos del arreglo ordenados son:  
Elemento [0]: 16  
Elemento [1]: 17  
Elemento [2]: 20  
Elemento [3]: 25  
Elemento [4]: 28  
Elemento [5]: 32

**2.- Genera un programa que le solicite una cadena de letras y números al usuario(emplear arreglo) e imprima solo letras. Este arreglo debe ser de caracteres, recuerda que un caracter puede almacenar números pero un arreglo de números no puede almacenar caracteres.**

Análisis:

DE: Imprimir solo las letras de un arreglo con una combinación de letras y números.

RE: Imprimir solo letras sin números, excluir(0,1,2,3,4,5,6,7,8,9).

DS: Obtener la impresión de las letras.

Ejemplo:

Entrada de usuario= Ej3mpl0c4d3n4

Salida en pantalla= Ejmplcdn

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#include <ctype.h> // Necesaria para la función isalpha
```

```
#define MAX_SIZE 100 // Definimos un tamaño máximo para el arreglo
```

```
int main() {
```

```
    // Declaración del arreglo de caracteres (cadena)
```

```
    char cadena[MAX_SIZE];
```

```
    int i;
```

```
    printf("--- Imprimir Solo Letras ---\n");
```

```
    printf("Ingrese una cadena de letras y números (máx. %d caracteres): ", MAX_SIZE - 1);
```

```
    // Leemos la cadena. %s lee hasta encontrar un espacio.
```

```
    scanf("%s", cadena);
```

```

printf("Salida en pantalla: ");

// Recorremos el arreglo de caracteres

for (i = 0; i < strlen(cadena); i++) {

    // La función isalpha() verifica si el carácter es una letra (a-z o A-Z)

    if (isalpha(cadena[i])) {

        printf("%c", cadena[i]); // Si es letra, la imprimimos

    }

}

printf("\n"); // Salto de línea al final

return 0;

}

```

The image shows a C program in a code editor (main.c) and its execution in a terminal. The code defines a constant MAX\_SIZE of 100, declares a character array 'cadena' and an integer 'i'. In the main function, it prints a separator, prompts the user to enter a string of letters and numbers (maximum 99 characters), reads the input using 'scanf', prints the output, and then iterates through the string to print each character. The terminal output shows the program running, the user entering 'Fra5n1cis7c09', and the program printing 'Salida en pantalla: Francisc' before finishing with exit code 0.

```

main.c
1 //Roja Martínez Francisco Alexander
2 //Tarea
3
4 #include <stdio.h>
5 #include <string.h>
6 #include <ctype.h> // Necesaria para la función isalpha
7
8 #define MAX_SIZE 100 // Definimos un tamaño máximo para el arreglo
9
10 int main() {
11     // Declaración del arreglo de caracteres (cadena)
12     char cadena[MAX_SIZE];
13     int i;
14
15     printf("--- Imprimir Solo Letras ---\n");
16     printf("Ingrese una cadena de letras y números (máx. %d caracteres): ", MAX_SIZE - 1);
17
18     // Leemos la cadena. %s lee hasta encontrar un espacio.
19     scanf("%s", cadena);
20
21     printf("Salida en pantalla: ");
22
23     // Recorremos el arreglo de caracteres

```

```

input
--- Imprimir Solo Letras ---
Ingrese una cadena de letras y números (máx. 99 caracteres): Fra5n1cis7c09
Salida en pantalla: Francisc

...Program finished with exit code 0
Press ENTER to exit console.

```

**3.- Genera un programa que solicite al usuario un vector de 15 enteros haciendo uso de un arreglo y la estructura iterativa para recorrerlo por medio de sus índices e imprimir en pantalla.**

```
#include <stdio.h>
```

```
#define VECTOR_SIZE 15 // Tamaño fijo del vector
```

```
int main() {
```

```
    // Declaración del arreglo de 15 enteros
```

```
    int vector[VECTOR_SIZE];
```

```
    int i; // Índice para recorrer el arreglo
```

```
    printf("--- Ingreso y Muestra de Vector de %d Enteros ---\n", VECTOR_SIZE);
```

```
    // Ciclo para solicitar los 15 elementos (Llenar el arreglo)
```

```
    // Se recorre por sus índices, desde 0 hasta 14 (VECTOR_SIZE - 1)
```

```
    for (i = 0; i < VECTOR_SIZE; i++) {
```

```
        printf("Ingrese el entero para el elemento [%d]: ", i);
```

```
        scanf("%d", &vector[i]);
```

```
    }
```

```
    printf("\nElementos del vector ingresados:\n");
```



```

// Ciclo para imprimir los 15 elementos (Mostrar el arreglo)

for (i = 0; i < VECTOR_SIZE; i++) {

    printf("Elemento [%d]: %d\n", i, vector[i]);

}

return 0;

}

```

The screenshot shows a C program in a code editor with a dark theme. The file is named 'main.c'. The code defines a constant 'VECTOR\_SIZE' as 15 and implements a 'main' function. Inside 'main', it declares an array 'vector' of size 'VECTOR\_SIZE' and an index 'i'. It prints a header message, then enters a loop to prompt the user for 15 integers, storing them in the 'vector' array. After the loop, it prints a message indicating the input is complete. The terminal window below the editor shows the program's execution, displaying the same header message and the first six prompts and inputs (2, 3, 5, 7, 7, 9).

```

main.c
1 //Roja Martínez Francisco Alexander
2 //Tarea
3
4 #include <stdio.h>
5
6 #define VECTOR_SIZE 15 // Tamaño fijo del vector
7
8 int main() {
9     // Declaración del arreglo de 15 enteros
10    int vector[VECTOR_SIZE];
11    int i; // Índice para recorrer el arreglo
12
13    printf("--- Ingreso y Muestra de Vector de %d Enteros ---\n", VECTOR_SIZE);
14
15    // Ciclo para solicitar los 15 elementos (llenar el arreglo)
16    // Se recorre por sus índices, desde 0 hasta 14 (VECTOR_SIZE - 1)
17    for (i = 0; i < VECTOR_SIZE; i++) {
18        printf("Ingrese el entero para el elemento [%d]: ", i);
19        scanf("%d", &vector[i]);
20    }
21
22    printf("\nElementos del vector ingresados:\n");
23

```

```

input
--- Ingreso y Muestra de Vector de 15 Enteros ---
Ingrese el entero para el elemento [0]: 2
Ingrese el entero para el elemento [1]: 3
Ingrese el entero para el elemento [2]: 5
Ingrese el entero para el elemento [3]: 7
Ingrese el entero para el elemento [4]: 7
Ingrese el entero para el elemento [5]: 9

```

4.- Modifica el programa (Programa7.c) y aplicar el uso de apuntadores.



```

#include <stdio.h>
#include <string.h>
int main()
{
    char palabra[20]; //arreglo tamaño 20
    int i=0;
    printf("Ingrese una palabra: ");
    scanf("%s", palabra); /* Se omite & porque el propio nombre del arreglo de
    tipo cadena apunta, es decir, es equivalente a la dirección de comienzo del
    propio arreglo*/
    printf("La palabra ingresada es: %s\n", palabra); // si fuera caracter sería %c
    // este for me sirve para imprimir la palabra que ingresó el usuario
    for (i = 0 ; i<strlen(palabra) ; i++) // i < 20    i<=strlen(palabra-1) (n-1)
    {
        printf("%c", palabra[i]); // %c porque va a recorrer letra por letra
    }
    return 0;
}

```

### Ya modificado

```

#include <stdio.h>

#include <string.h>


int main() {

    char palabra[20]; // Arreglo tamaño 20

    char *ptrPalabra; // ;Declaración del apuntador a char!

    int i;

```

```
printf("--- Uso de Apuntadores en Cadena ---\n");

printf("Ingrese una palabra (máx. 19 letras): ");

// La lectura de la cadena no cambia: 'palabra' es la dirección base
scanf("%s", palabra);

// Asignación de la dirección base del arreglo al apuntador
ptrPalabra = palabra;

printf("La palabra ingresada es: %s\n", palabra);

printf("Impresión letra por letra con apuntadores:\n");

// Recorremos la cadena usando strlen para el límite
// Nota: El apuntador NO se incrementa en el ciclo (ptrPalabra + i)
for (i = 0 ; i < strlen(palabra) ; i++) {

    // Accedemos al carácter en la posición i usando aritmética de apuntadores
    // Equivale a printf("%c", palabra[i]);

    printf("%c", *(ptrPalabra + i));

}
```

```

printf("\n");

return 0;

}

```

```

1 //Roja Martínez Francisco Alexander
2 //Tarea
3 |
4 #include <stdio.h>
5 #include <string.h>
6
7 int main() {
8     char palabra[20]; // Arreglo tamaño 20
9     char *ptrPalabra; // ¡Declaración del apuntador a char!
10    int i;
11
12    printf("--- Uso de Apuntadores en Cadena ---\n");
13    printf("Ingrese una palabra (máx. 19 letras): ");
14
15    // La lectura de la cadena no cambia: 'palabra' es la dirección base
16    scanf("%s", palabra);
17
18    // Asignación de la dirección base del arreglo al apuntador
19    ptrPalabra = palabra;
20
21    printf("La palabra ingresada es: %s\n", palabra);
22
23    printf("Impresión letra por letra con apuntadores:\n");

```

input

```

La palabra ingresada es: Dormir
Impresión letra por letra con apuntadores:
Dormir

...Program finished with exit code 0
Press ENTER to exit console.

```

## Conclusion

Esta práctica resultó ser más complicada de lo esperado, aunque no al nivel de algunas anteriores. La mayor dificultad provino del manejo de los apuntadores, un tema que aún no domino completamente y que sigue siendo mi principal desafío en C.

A pesar de la complejidad, me esforcé por llevar a cabo la práctica. Para compensar mis dudas, realicé investigaciones y estudios adicionales por mi cuenta, lo cual fue

clave para entender cómo los apuntadores interactúan con los arreglos y cómo implementar la aritmética de apuntadores en el código (Tarea 4).

Aunque persisten las dudas sobre la perfección de mis implementaciones, estoy satisfecho de haber logrado completar todos los ejercicios e integrar los nuevos conceptos. Este esfuerzo extra me ha permitido avanzar significativamente en la comprensión de la relación fundamental entre arreglos y memoria, lo cual es esencial para programar eficientes.

### ***Referencia***

***Facultad de Ingeniería. (2025). Manual de prácticas del laboratorio de Fundamentos de Programación. Laboratorio de computación. Salas A y B. Universidad Nacional Autónoma de México. pp.135-148 . Recuperado el 21 de octubre de 2025 de [Laboratorio Salas A y B](#)***